9.1 Singular Value Decomposition

In this section, we deal with one of the most important matrix factorization tools, called the singular value decomposition (SVD). The SVD of a matrix A is closely related to eigen decomposition of the matrix AA^T. One can also think of this as a generalization of diagonalization procedure that allows us to diagonalize any matrix not necessarily square matrix. The SVD is computationally a viable tool for a wide variety of applications. It has applications in image and signal processing, control theory, least square problems, time series analysis, pattern recognition, dimensionality reduction, biomedical engineering and defining a generalized inverse of a matrix and many more. We shall deal with few of these applications.

9.1.1 Singular Value Decomposition Theorem

Theorem 9.1.1. SVD Theorem. Let A be a real $m \times n$ matrix. Then A can be factorized as

$$A = U \sum V^T \tag{9.1.1}$$

where U is an $m \times m$ orthogonal matrix, V is an $n \times n$ orthogonal matrix and $\sum$ is a $m \times n$ diagonal matrix given by

$$\left[\begin{array}{ccc|ccc} \sigma_1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r & 0 & \cdots & 0 \\ \hline 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & 0 & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{array}\right]_{m \times n}$$

whose diagonal entries are non negative and are arranged in a non increasing order.

The number of non zero entries in $\sum$ is the rank of A.

The decomposition $A = U \sum V^T$ is called a *singular value decomposition* of A. The diagonals entries $\sigma_1, \sigma_2, \ldots, \sigma_r$ are called *singular values* of A. (that is why the name singular value decomposition.)

Before we prove this theorem, let us play with the equation (9.1.1). We have

$$A^T A = \left(V \Sigma^T U\right)\left(U^T \Sigma V^T\right) = V \Sigma^T \Sigma V^T.$$

This implies

$$V^T (A^T A) V = \Sigma^T \Sigma.$$

Hence

$$V^T(A^TA)V = \left[\begin{array}{ccc|ccc} \sigma_1^2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r^2 & 0 & \cdots & 0 \\ \hline 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & 0 & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{array}\right]_{n\times n} \tag{9.1.2}$$

The Eqn. (9.1.2) suggests that columns of V are eigenvectors of A^TA, and are called *right singular vectors*.

Similarly,

$$AA^T = \left(U\Sigma V^T\right)\left(V\Sigma^T U^T\right) = U\Sigma\Sigma^T U^T$$

This implies

$$U^T(AA^T)U = \left[\begin{array}{ccc|ccc} \sigma_1^2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r^2 & 0 & \cdots & 0 \\ \hline 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & 0 & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{array}\right]_{m\times m} \tag{9.1.3}$$

The Eqn. (9.1.3) suggests that columns of U are eigenvectors of AA^T, and are called *left singular vectors*.

The notion of right and left eigenvectors suggest a way to construct the matrix V and U in the SVD decomposition. Let us see what do I mean? Suppose $U = [u_1\ u_2\ \cdots\ u_m]$ and $V = [v_1\ v_2\ \cdots\ v_n]$. Then we have

$$A^TAv_i = \sigma_i^2 v_i \text{ and } AA^Tu_i = \sigma_i^2 u_i.$$

Hence

$$A(A^TAv_i) = (AA^T)(Av_i) = A(\sigma_i^2 v_i) = \sigma_i^2(Av_i).$$

Thus suggests, that u_i may be defined as Av_i. However, if $u_i = Av_i$, then

$$\langle u_i, u_j\rangle = \langle Av_i, Av_j\rangle = \left\langle A^TAv_i, v_j\right\rangle = \sigma_i^2\langle v_i, v_j\rangle = \sigma_i^2\delta_{ij}.$$

If we want U to be orthogonal, we may define $u_i = \frac{1}{\sigma_i}Av_i$.

Let us look at proof of Theorem 9.1.1.

▶ **Proof.**

Remark 9.1.2. The singular values of a matrix in a SVD are unique, however singular vectors are not unique.

Corollary 9.1.3. Let $A = U\Sigma V^T$ be a SVD of A where $U = [u_1\ u_2\ \cdots\ u_m]$ and $V = [v_1\ v_2\ \cdots\ v_n]$, $\sigma_1, \ldots, \sigma_r$ are singular values of A. Then A can also be written as

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T. \tag{9.1.7}$$

The decomposition (9.1.7) is called a *rank one decomposition of* A, as rank of each term in (9.1.7) is 1. (why?) This is a very useful way of decomposing A as we shall see this later.

Reading Questions

1. Use SVD to show that a square matrix A is symmetric ($A^T = A$) if and only if $A^TA = AA^T$.

▶ Hint.

Exercises

1. An $n \times n$ matrix A is rank one matrix if and only if there exist non-zero vectors $p, q \in \mathbb{R}^n$ such that $A = pq^T$.

2. If A is a real symmetric matrix with eigenvalues $\lambda_1, \ldots, \lambda_n$, then show that singular values of A are $|\lambda_1|, \ldots, |\lambda_n|$

3. A square matrix A is non singular if and only if all singular values of A are non zero.

Example 9.1.4. Let us find a singular value decomposition of $A = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}$.

We have $A^TA = \begin{pmatrix} 13 & 12 & 2 \\ 12 & 13 & -2 \\ 2 & -2 & 8 \end{pmatrix}$. The eigenvalues of A^TA are $\sigma_1^2 = 25, \sigma_2^2 = 9$ and $\sigma_3^2 = 0$. The corresponding eigenvectors with respect to eigenvalues $25, 9, 0$ of A^TA are $\begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$, $\begin{pmatrix} 1 \\ -1 \\ 4 \end{pmatrix}$ and

$\begin{pmatrix} 1 \\ -1 \\ -\frac{1}{2} \end{pmatrix}$ respectively. Hence an orthonormal eigenbasis of A^TA is

$$\{v_1, v_2, v_3\} = \left\{ \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \\ 0 \end{pmatrix}, \begin{pmatrix} 1/\sqrt{18} \\ -1/\sqrt{18} \\ 4/\sqrt{18} \end{pmatrix}, \begin{pmatrix} 2/3 \\ -2/3 \\ -1/3 \end{pmatrix} \right\}.$$

We define

$$u_1 := \frac{1}{\sigma_1} Av_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} \text{ and } u_2 := \frac{1}{\sigma_2} Av_2 = \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}.$$

Thus we have

$$U = \begin{pmatrix} \frac{1}{2}\sqrt{2} & \frac{1}{2}\sqrt{2} \\ \frac{1}{2}\sqrt{2} & -\frac{1}{2}\sqrt{2} \end{pmatrix}$$

$$V = \begin{pmatrix} \frac{1}{2}\sqrt{2} & \frac{1}{6}\sqrt{2} & \frac{2}{3} \\ \frac{1}{2}\sqrt{2} & -\frac{1}{6}\sqrt{2} & -\frac{2}{3} \\ 0 & \frac{2}{3}\sqrt{2} & -\frac{1}{3} \end{pmatrix} \text{ and } \sum = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix}.$$

It is easy to check that that $UAV^T = \sum$.

```
A = matrix([[3,2,2],[2,3,-2]])
print('A^TA is:' )
ATA = A.T*A
print(ATA)
print('Eigenvalues of A^TA are:')
evals = ATA.eigenvalues()
print(evals)
V=[]
evecs = ATA.eigenvectors_right()
for i in range(len(evecs)):
    print(f'The eigenvector with respect to eigenvalue {evecs[i][0]} is {evecs[i]
```

Evaluate (Sage)

Sage includes a built-in method for computing the SVD of a matrix. However, the matrix must be defined over the Real Double Field (RDF), a numerical field of real numbers represented in 64-bit double precision, offering approximately 15–16 decimal digits of accuracy. Let us see to achieve this.

```
A = matrix(RDF,[[3,2,2],[2,3,-2]])
# Compute the SVD
U, S, V = A.SVD()
# Display results
print("U =\n", U)
print("Singular values =\n", S)
print("V =\n", V)
```

Evaluate (Sage)

```
# Reconstruction
A_reconstructed = U * S * V.transpose()
print(A_reconstructed)
print('Due to numeruical computation,we shall check if the norm of the difference is
print(norm(A_reconstructed.numerical_approx()-A.numerical_approx()))
```

Evaluate (Sage)

Example 9.1.5. Find the signular value decomposition of the matrix $A = \begin{pmatrix} 3 & 5 \\ 5 & -3 \\ 4 & 4 \end{pmatrix}$.

▶ Solution.

Exercises

1. Verify the equation (9.1.7) for example 9.1.4. That is,

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T.$$

2. Find a singular value decomposition of $A = \begin{pmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \end{pmatrix}$.

3. Find the SVD of $A = \begin{pmatrix} 1 & 1 \\ 2 & 2 \\ 3 & 1 \end{pmatrix}$ using step by step calculations.

9.1.2 Pseudoinverse using SVD

Definition 9.1.6. If A is a $m \times n$ matrix, then the ***pseudoinverse*** of A is matrix X satisfying the following properties:

1. $AXA = A$
2. $XAX = X$

3. $(AX)^T = AX$
4. $(XA)^T = XA.$

The pseudoinverse of a matrix A is denoted by $A^\dagger$. Pseudoinverse is also called the ***generalized inverse*** or ***Moore-Penrose pseudoinverse.***

Singular value decomposition provides an effective procedure to find the pseudoinverse of a matrix.

Suppose $A = U\sum V^T$ is a SVD of A. Since U and V are orthogonal matrices they are invertible. Thus to define pseudoinverse of A, it is sufficient to define pseudoinverse of the diagonal matrix $\sum$. It is natural to define the inverse of diagonal matrix by taking reciprocal of the nonzero diagonal entries and taking its transpose. Thus if

$$\Sigma = \left[\begin{array}{ccc|ccc} \sigma_1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r & 0 & \cdots & 0 \\ \hline 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & 0 & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{array}\right]_{m\times n}.$$

Then

$$\Sigma^\dagger = \left[\begin{array}{ccc|ccc} 1/\sigma_1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 1/\sigma_r & 0 & \cdots & 0 \\ \hline 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & 0 & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{array}\right]^T_{m\times n}.$$

Having defined the generalized inverse of $\sum$, now it is natural to define

$$A^\dagger := V\sum{}^\dagger U^T. \tag{9.1.8}$$

Example 9.1.7. Find the pseudoinverse of $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & -1 \end{pmatrix}$.

Note that $A^TA = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$. The eigenvalues of A^TA are $\sigma_1^2 = 4$ and $\sigma_2^2 = 2$ with corresponding orthonormal eigenvectors $v_1 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$ and $v_2 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$ respectively. Now

$$u_1 = \frac{1}{\sigma_1} Av_1 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix} \text{ and } u_2 = \frac{1}{\sigma_2} Av_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

Extending u_1, u_2 to an orthonormal basis of $\mathbb{R}^3$, we can select $u_3 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \\ 0 \end{pmatrix}$.

Thus a SVD is given by

$$A = U \sum V^T = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & 0 & -\frac{1}{\sqrt{2}} \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & \sqrt{2} \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}.$$

Hence

$$\begin{aligned} A^\dagger = V \sum{}^\dagger U^T &= \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 1/2 & 0 & 0 \\ 0 & \frac{1}{\sqrt{2}} & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \end{pmatrix} \\ &= \begin{pmatrix} 1/4 & 1/4 & 1/2 \\ 1/4 & 1/4 & -1/2 \end{pmatrix}. \end{aligned}$$

Sage provdes a method `.pseudoinverse()` to find the pseudoinverse of a matrix.

```
# pseudoinverse using inbuilt Sage method
A = matrix(SR,[[1,1],[1,1],[1,-1]])
A.pseudoinverse()
```



Evaluate (Sage)

```
## pseudoinverse using SVD
A = matrix(SR,[[1,1],[1,1],[1,-1]])
A1 = A.change_ring(RDF)
U,S, V = A1.SVD()
# To find generatlize inverse of S
S1 = S.T
for i in range(2):
    S1[i,i]=1/S1[i,i]
V*S1*U.T
```



Evaluate (Sage)

Reading Questions

1. If A is $m \times n$ matrix with $m \geq n$ and $rank\,(A) = n$ then

$$A^{\dagger} = (A^T A)^{-1} A^T.$$

$(A^T A)^{-1} A^T$ is called the *left pseudoinnverse* of A.

2. If $m \leq n$ and $rank\,(A) = m$ then

$$A^{\dagger} = A^T (A A^T)^{-1}.$$

$A^T (A A^T)^{-1}$ is called the *right pseudoinverse* of A.

Theorem 9.1.8. Let $Ax = b$, be a system of m equations in n variables with $m \geq n$. Let $rank\,(A) = n$. Then the vector $x^* = A^{\dagger} b$ minimizes $\|Ax - b\|^2$ on $\mathbb{R}^n$, that is the least square solution of $Ax = b$.

Example 9.1.9. Example 8.1.7

▶ Solution.

Exercises

1. If A is square matrix, then show that $A^T A$ and $A A^T$ are similar.

2. Find the SVD of a matrix $\begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 1 & -1 \end{pmatrix}$.

3. Find the least square solution of the system of equations $Ax = b$ where $A = \begin{pmatrix} 2 & 3 & -1 \\ -2 & 1 & 4 \\ 3 & 1 & 3 \\ -5 & 4 & 2 \\ 1 & 1 & 1 \end{pmatrix}$ and $b = \begin{pmatrix} -10 \\ 7 \\ 15 \\ 8 \\ 9 \end{pmatrix}$ using generalized inverse.

9.1.3 Geometry of SVD

SVD provides effective way to look at how a matrix tranforms and object geometrically. In order to see the geometric, let us consider the matrix $A = \begin{pmatrix} 4 & -2 \\ -2 & 3 \end{pmatrix}$.

The singular value decomposition of $A = USV^T$ is given by

$$A = \begin{pmatrix} -0.78820 & 0.61541 \\ 0.61541 & 0.78820 \end{pmatrix} \begin{pmatrix} 5.56150 & 0.0 \\ 0.0 & 1.4384 \end{pmatrix} \begin{pmatrix} -0.78820 & 0.61541 \\ 0.61541 & 0.78820 \end{pmatrix}^T.$$

The geometric meaning of A applied to unit circle along with unit vectors e_1 and e_2 is explained in the following figure Figure 9.1.10.

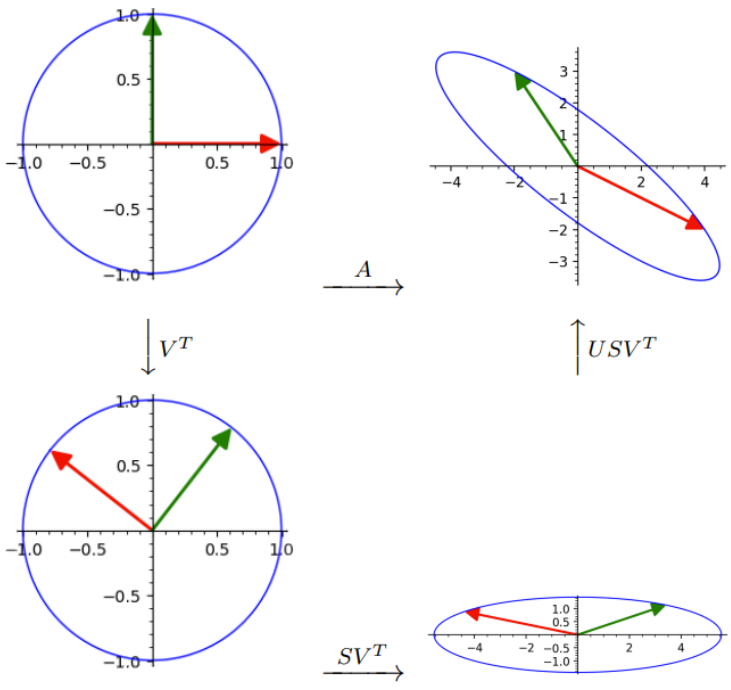


Figure 9.1.10. Transformation of unit circle under SVD

Let us use Sage to demonstrate geometry of the above example.

```
A=matrix(RDF,[[4,-2],[-2,3]])
U,S,V=A.change_ring(RDF).SVD()
```

Evaluate (Sage)

```
var('t')
cir=vector([cos(t),sin(t)])
c1=parametric_plot(cir,(t,0,2*pi),figsize=4)
e1 =vector([1,0])
e2 =vector([0,1])
svd1=c1+plot(e1,color='red')+plot(e2,color='green')
svd1
```

Evaluate (Sage)

```
c2=parametric_plot(V.T*cir,(t,0,2*pi),figsize=4)
svd2=plot(V.T*e1,color='red')+plot(V.T*e2,color='green')+c2
show(svd2)
```

Evaluate (Sage)

```
c3=parametric_plot(S*V.T*cir,(t,0,2*pi),figsize=4)
svd3=plot(S*V.T*e1,color='red')+plot(S*V.T*e2,color='green')+c3
svd3.show()
```

Evaluate (Sage)

```
c4=parametric_plot(U*S*V.T*cir,(t,0,2*pi),figsize=4)
svd4=plot(U*S*V.T*e1,color='red')+plot(U*S*V.T*e2,color='green')+c4
svd4.show()
```

Evaluate (Sage)

We can plot the image under A.

```
## Image under A
svdCir = parametric_plot(A*cir,(t,0,2*pi),figsize=4)
svdA=plot(A*e1,color='red')+plot(A*e2,color='green')+c4
show(svdA+svdCir)
```

Evaluate (Sage)

Example 9.1.11. Consider a 3×3 matrix $A = \begin{pmatrix} 2 & 3 & 1 \\ -1 & 2 & 1 \\ 0 & 2 & 3 \end{pmatrix}$. The singular values of A are 5.107, 2.2982 and 1.2779.

The Figure Figure 9.1.12 below explains what happens to a unit sphere and unit vectors under A, obtained using SVD.

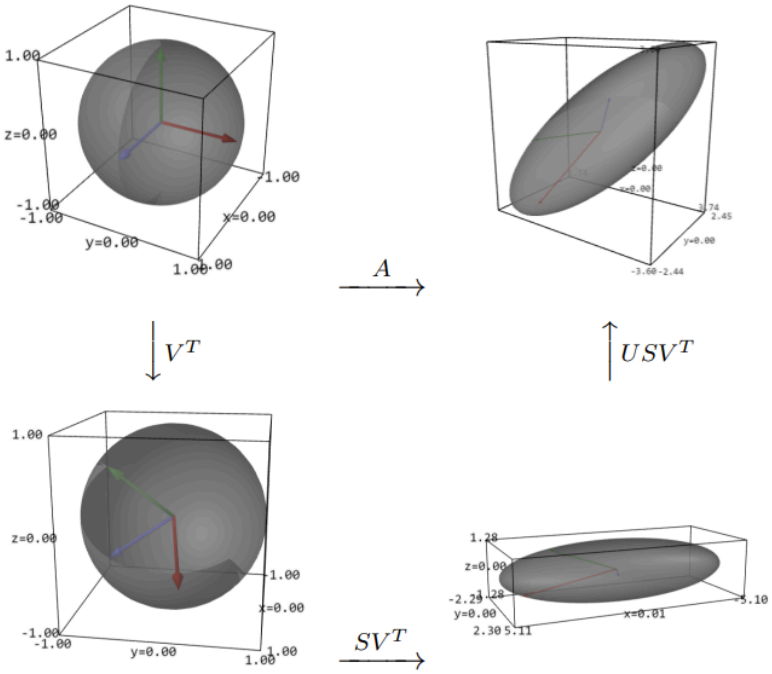


Figure 9.1.12. Transformation of unit circle under SVD

We can also demostrate the 3×3 example also in Sage.

```
A=matrix(RDF, [[2,3,1],[-1,2,1],[0,2,3]])
U, S, VT = A.SVD()
V=VT.transpose()
v1=V.columns()[0]
v2=V.columns()[1]
v3=V.columns()[2]
```

Evaluate (Sage)

```
## parametric coordinates of shpere
var('s,t')
spr=vector([cos(s)*sin(t),sin(s)*sin(t),cos(t)])
e = identity_matrix(3).columns()
P0=parametric_plot(spr,(s,0,2*pi),(t,0,pi),igsize=2,color='gray',opacity=0.3)
cord_axes=plot(e[0],thickness=0.5)+plot(e[1],color='red',thickness=0.5)+\
plot(e[2],color='green',thickness=0.5)
P0=P0+cord_axes
P0.show(aspect_ratio=[1,1,1])
```

Evaluate (Sage)

```
P1=parametric_plot(spr,(s,0,2*pi),(t,0,pi),color='gray',opacity=0.7)+\
plot(e[0],thickness=2)+plot(e[1],color='red',thickness=2)+plot(e[2],color='green',thi
P1 = P1+cord_axes
P1.show(aspect_ratio=[1,1,1])
```

Evaluate (Sage)

```
P2=parametric_plot(VT*spr,(s,0,2*pi),(t,0,pi),igsize=4,color='gray',opacity=0.7)+\
plot(VT*e[0],thickness=2)+plot(VT*e[1],color='red',thickness=2)+plot(VT*e[2],color='g
P2.show(aspect_ratio=[1,1,1])
```

Evaluate (Sage)

```
P3=parametric_plot(S*V.transpose()*spr,(s,0,2*pi),(t,0,pi),igsize=4,color='gray',ppac
plot(S*VT*e[0],thickness=2)+plot(S*VT*e[1],color='red',thickness=2)+plot(S*VT*e[2],co
P3.show(aspect_ratio=[1,1,1])
```

Evaluate (Sage)

```
P4=parametric_plot(U*S*VT*spr,(s,0,2*pi),(t,0,pi),igsize=4,color='gray',opacity=0.7)+
plot(U*S*VT*e[0],thickness=2)+plot(U*S*VT*e[1],color='red',thickness=2)+\
plot(U*S*VT*e[2],color='green',thickness=2)
P4.show(aspect_ratio=[1,1,1])
```

Evaluate (Sage)

9.1.4 Image Compression using SVD

Images stored on a computer is a collection of dots called pixels. The collection of dots/pixels that constitute an image can be stored as a matrix. The color image can be thought of as 3 dimensional array.

Using Eqn. (9.1.7) we can write a matrix as sum of rank one matrices.

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T.$$

This property says that $rank\,(A)$ is equal to the number of singular values of A. Since $\sigma_1 > \sigma_2 > \cdots > \sigma_r$, the first term has highest impact on A followed by the second term and so on. This propriety allows us to reduce the noise or compress the matrix data by eliminating the small singular values or the higher ranks. This can be used as approximation of a given matrix, in particular we can approximate a matrix by adding only the first few terms of (9.1.7).

If we let

$$A_k := \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_k u_k v_k^T (k \leq r)$$

then the total storage required for A_k is $k(m+n+1)$ which is much less compare to mn.

When an image (the corresponding matrix) is transformed using SVD, it is not compressed, but the data take a form in which the first singular value has a more amount of the image information. This allows us to use first few singular values to represent the image almost identical to the original.

Look at the Image in Fig 9.1.13. The associates matrix for this image is of size $\mathbf{995 \times 1770 \times 3.}$



Figure 9.1.13. Orginal Color Image

This image is converted into a gray scale image (See Figure 9.1.14). This size of matrix associated to the gray scale image is $\mathbf{995 \times 1770}$ with rank 995.

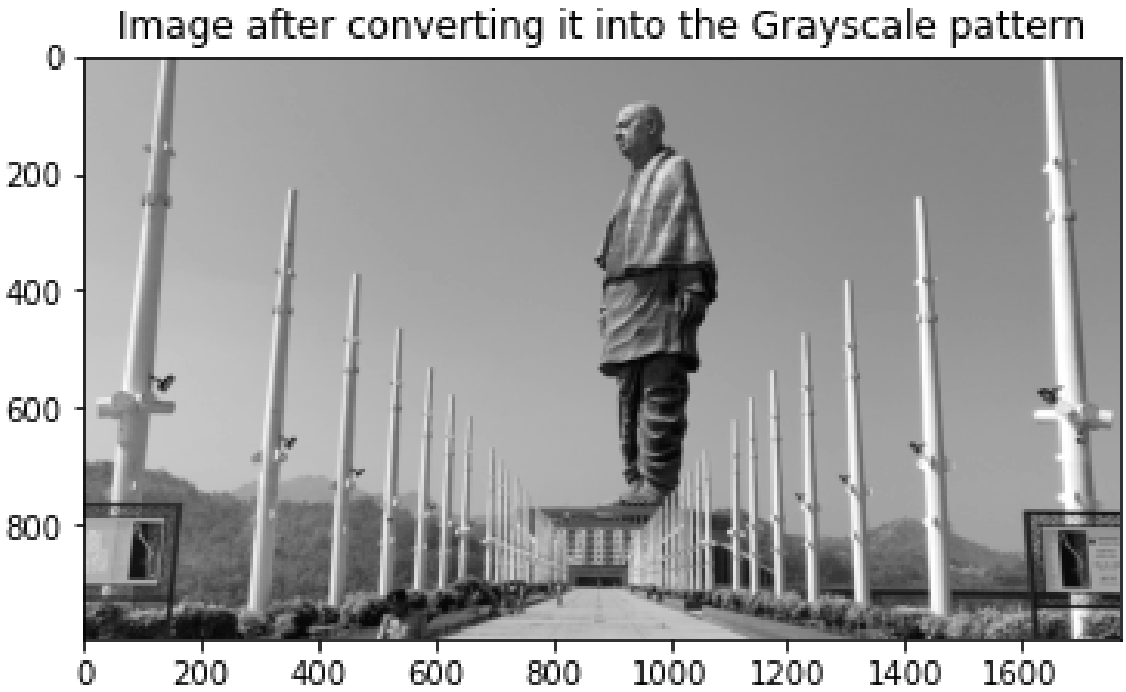


Figure 9.1.14. Original Gray Image

After applying SVD to the Gray image and using 1st 5, 10, 20,30, 50, 100 terms respectively of the rank one approximation the approximate images are plotted in the Figures 9.1.15, Figure 9.1.16, Figure 9.1.17, Figure 9.1.19, Figure 9.1.19, Figure 9.1.20 respectively.

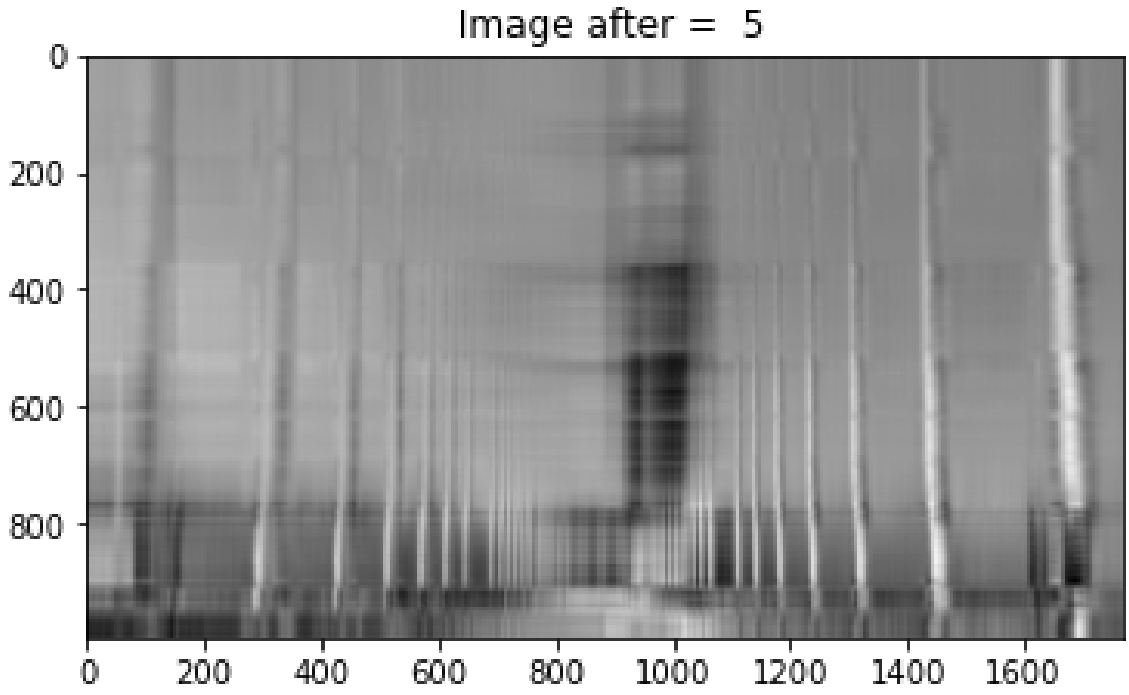


Figure 9.1.15. Approximate Image with 5 terms

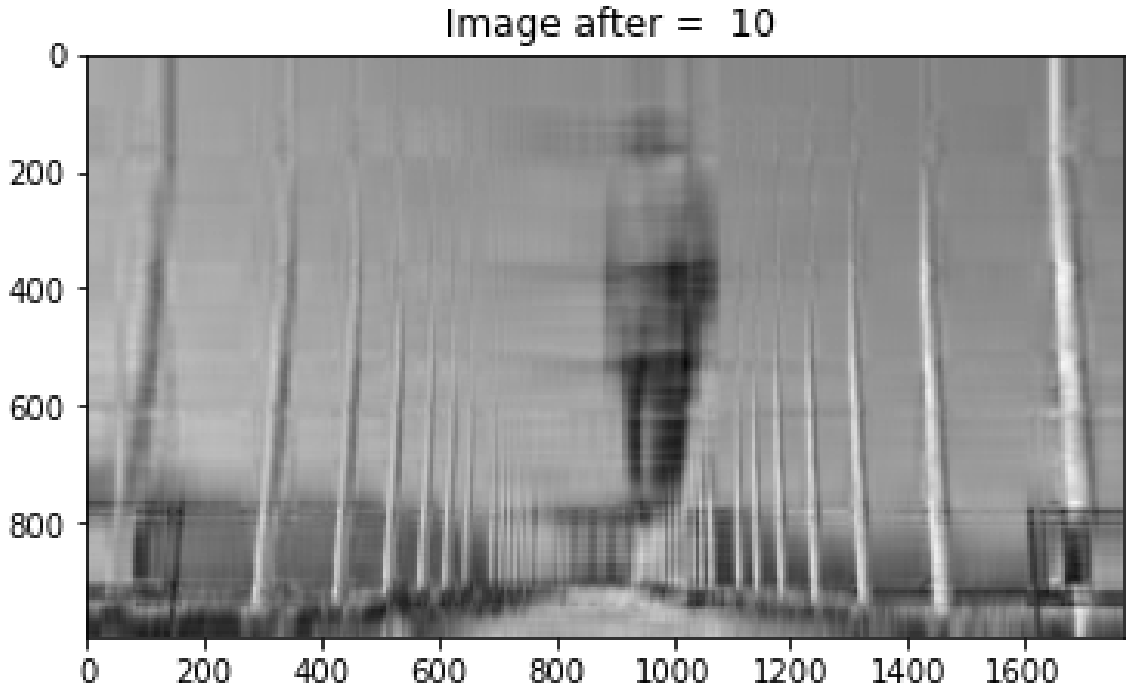


Figure 9.1.16. Approximate Image with 10 terms

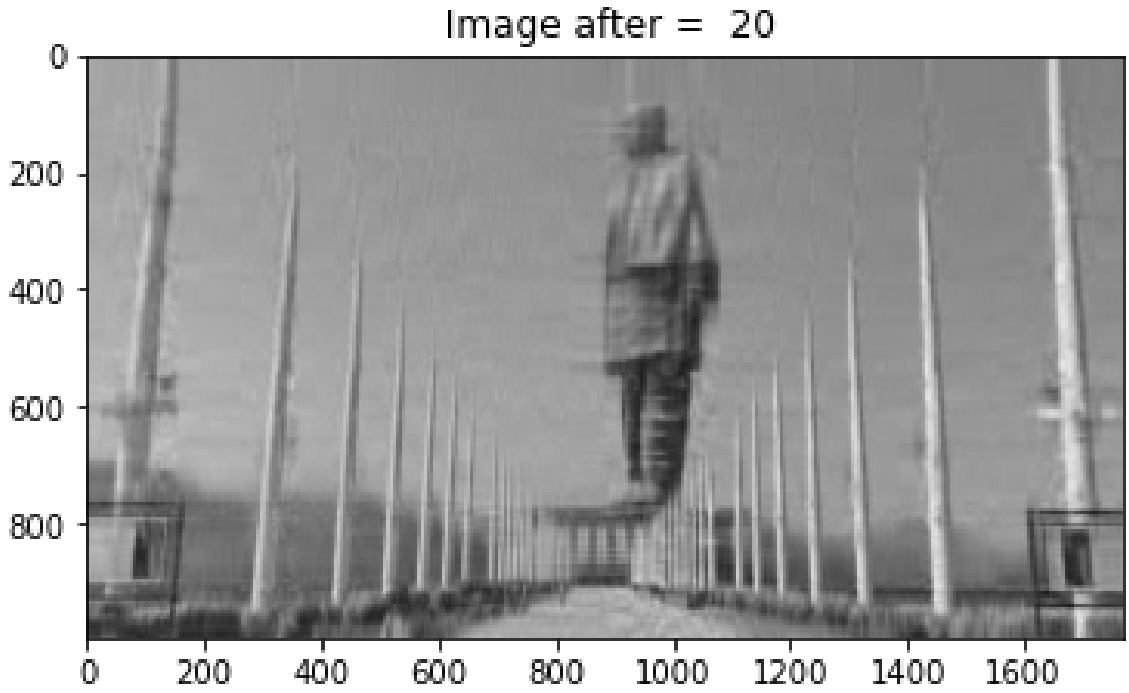


Figure 9.1.17. Approximate Image with 20 terms

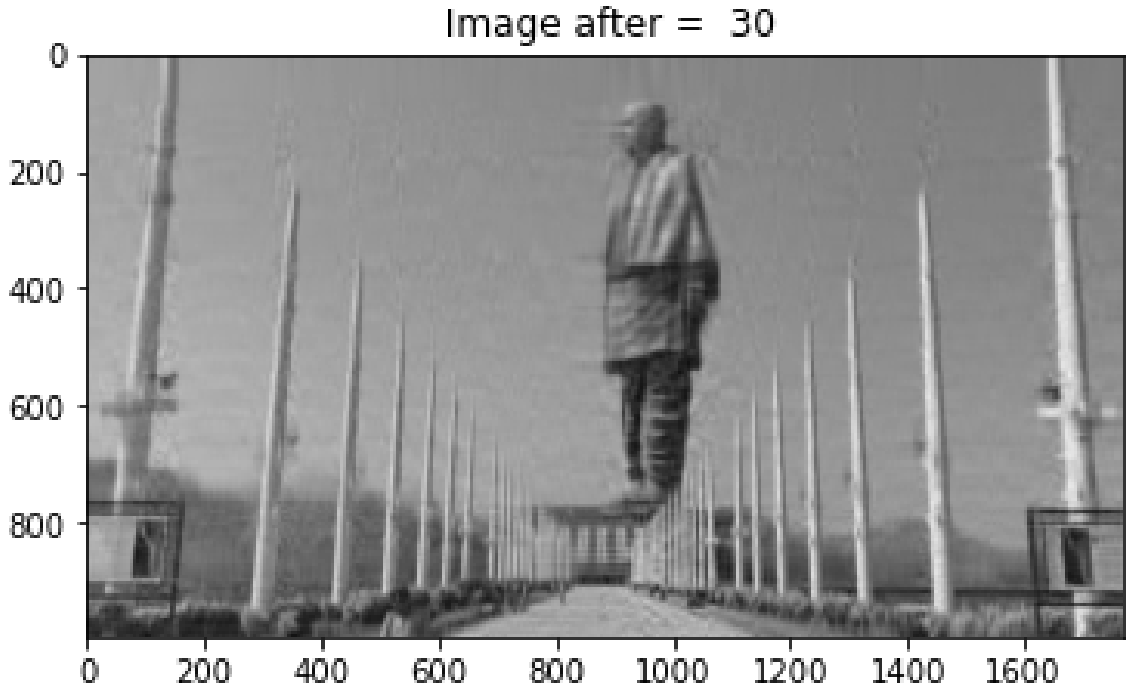


Figure 9.1.18. Approximate Image with 30 terms

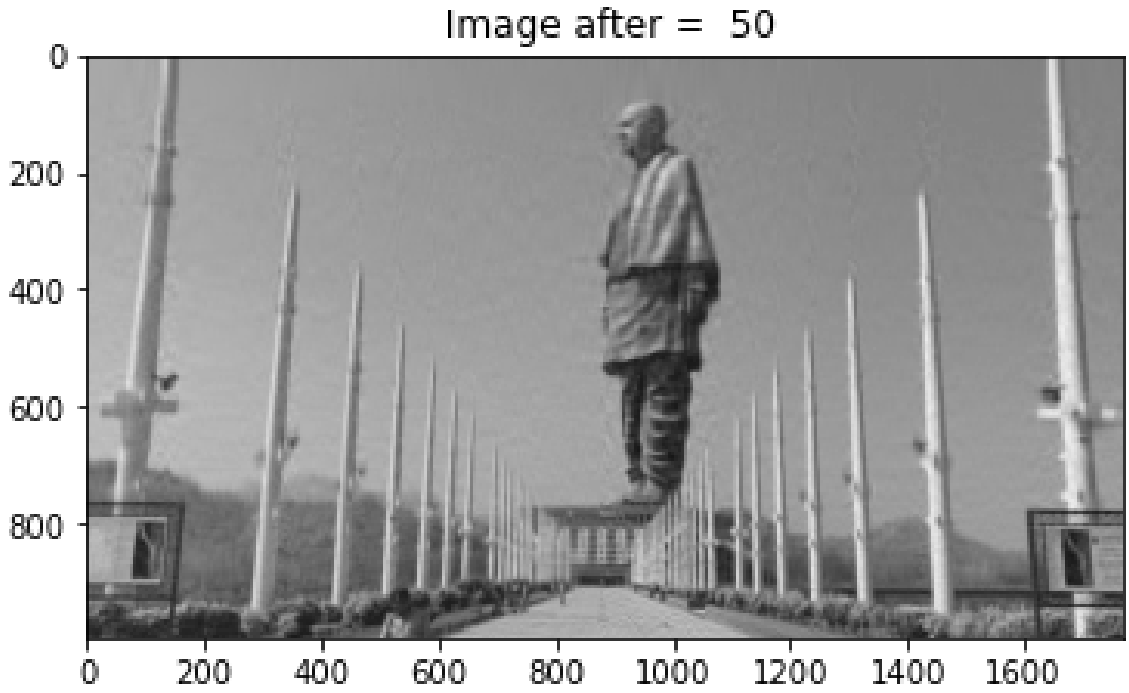


Figure 9.1.19. Approximate Image with 50 terms

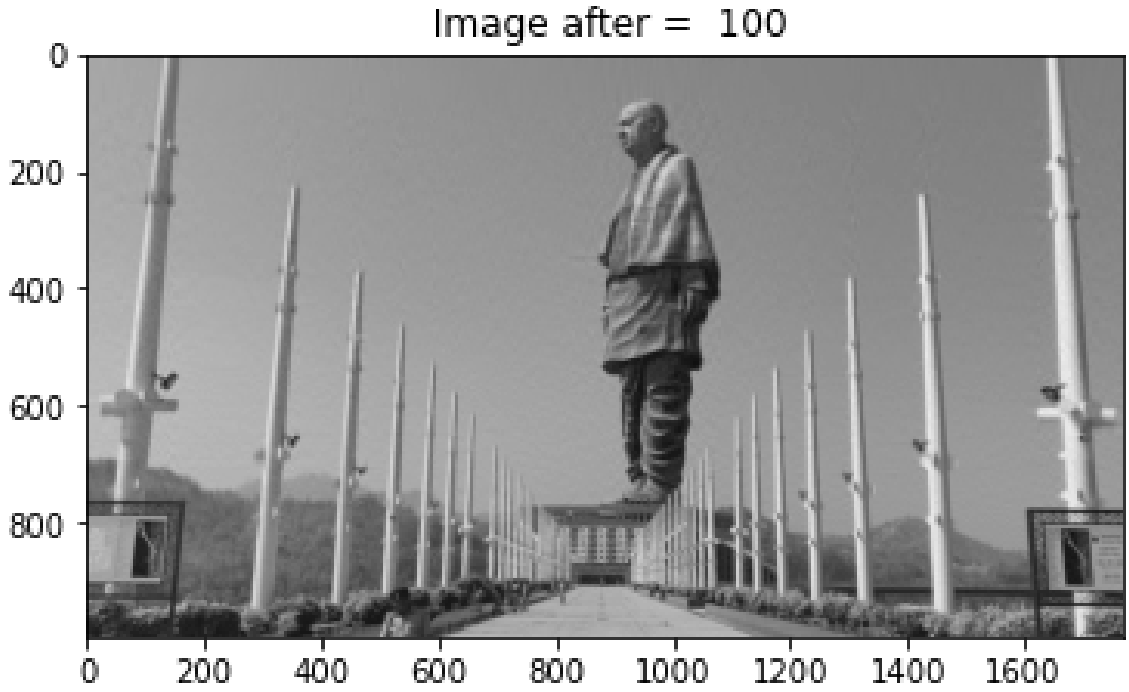


Figure 9.1.20. Approximate Image with 100 terms

It is quite evident that 1st 100 terms itself gives a very good approximation of the original gray scale image. Note that the original image has $995 \times 1770 = 1761150$ pixels, where as if we take the 1st 100 terms, then it is of size $100(995 + 1770) = 276500$ which quite small compared to the original size.

Now let us see how, we can implement image compression in Python.

```
import numpy as np
from PIL import Image
import requests
from io import BytesIO
import matplotlib.pyplot as plt
url ="https://raw.githubusercontent.com/ajitbkp/LinAlg-SageBook/main/assets/images/Sa
response = requests.get(url)
img = Image.open(BytesIO(response.content))
img_array = np.array(img)
# img.show() # may open in viewer depending on Sage environment
img
```

Evaluate (Sage)

```
imggray = img.convert('LA')
imgmat = np.array( list(imggray.getdata(band = 0)), float)
imgmat.shape = (imggray.size[1], imggray.size[0])
imgmat.shape
```

Evaluate (Sage)

```
plt.figure()
plt.imshow(imgmat, cmap = 'gray')
plt.title("Image after converting it into the Grayscale pattern")
plt.show()
```

Evaluate (Sage)

```
U, S, Vt = np.linalg.svd(imgmat) #singular value decomposition
```

Evaluate (Sage)

```
i = 10
cmpimg = np.matrix(U[:, :i]) * np.diag(S[:i]) * np.matrix(Vt[:i,:])
plt.imshow(cmpimg, cmap = 'gray')
title = " Image after =  %s" %i
plt.title(title)
plt.show()
```

Evaluate (Sage)

```
print("After compression: ")
for i in [5, 10, 50,100,200]:
    cmpimg = np.matrix(U[:, :i]) * np.diag(S[:i]) * np.matrix(Vt[:i,:])
    plt.imshow(cmpimg, cmap = 'gray')
    title = "Image with %s components" %i
    plt.title(title)
    plt.show()
    result = Image.fromarray((cmpimg ).astype(np.uint8))
```

Evaluate (Sage)

Example 9.1.21. Color Image Compression. In this example, we present a Python program that demonstrates color image compression using Singular Value Decomposition (SVD). The method involves separating the image into its Red, Green, and Blue (RGB) channels, applying SVD to each channel individually, and then recombining the compressed channels to obtain the final compressed image.

The reader is encouraged to copy the following code and execute it in a Python environment.

```
import numpy as np
from PIL import Image
import requests
from io import BytesIO
import matplotlib.pyplot as plt

# Load the image from the URL
url = "https://raw.githubusercontent.com/ajitbkp/LinAlg-SageBook/main/assets/imag
response = requests.get(url)
img = Image.open(BytesIO(response.content))
img = np.array(img, dtype=float)
```

Evaluate (Sage)

Remark 9.1.22. Suppose $A = U\Sigma V^T$ is the singular value decomposition of A. Then we get the following informations from the SVD of A.

1. The rank is equal to the number of non-zero singular values: $\mathbf{rank}(A) = \#\{\sigma_i > 0\}$
2. Null space (Kernel) is obtained from the columns of V corresponding to zero singular values.
3. Column space (Range) is panned by the first r columns of U, where $r = \mathbf{rank}(A)$.
4. Row space is spanned by the first r columns of V.
5. Spectral norm, $\|A\|_2 = \sigma_{\mathbf{max}}$
6. Frobenius norm $\|A\|_F = \sum a_{ij}^2 = \sqrt{\sum \sigma_i^2}$
7. Condition number, defined as $\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$.

In essence, SVD bridges theory and application—providing a unifying framework in linear algebra with far-reaching uses from solving least squares problems to dimensionality reduction and real-world data compression.